FoodDash

`com.fooddash.app`

Sample report for a fictional app. File: fooddash-4.12.0.apk

VERSION	ANDROID	SIZE	STACK
4.12.0 (41200)	**API 24-35**	**36.6 MB**	**Native Android**

19 SDKs and libraries · 4 sensitive permissions · 5 entry points · 3 hosts · 8 flows (83 steps)

Built with

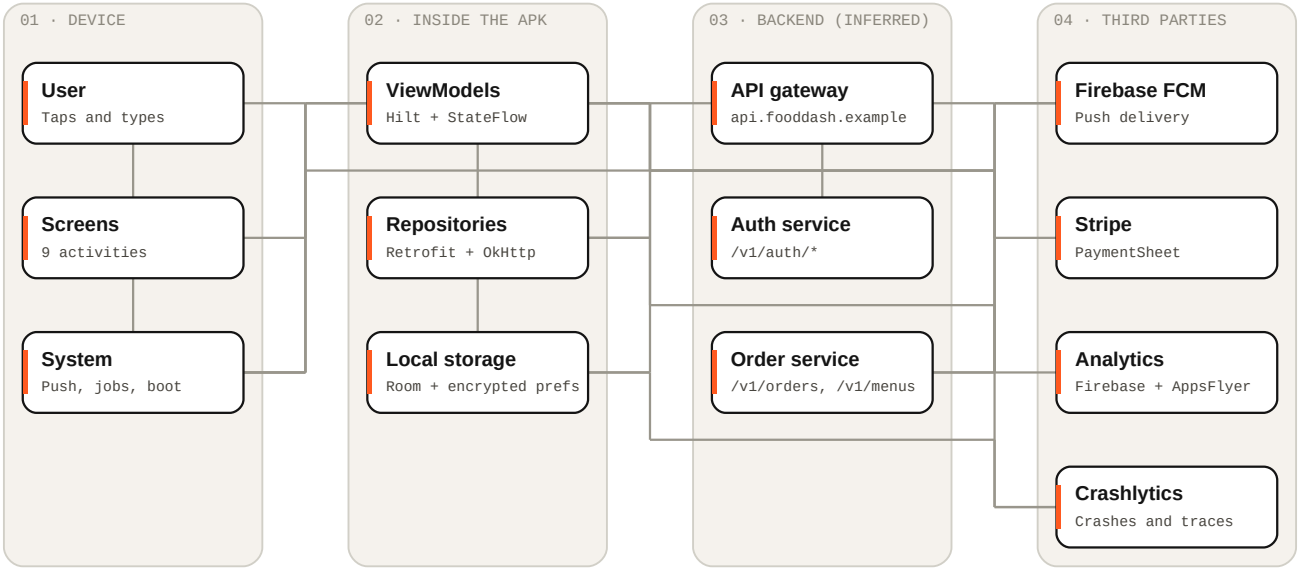
Native Android · Kotlin

- Kotlin standard library
- Views + Jetpack Navigation
- Native libraries for arm64-v8a, armeabi-v7a

CPU architectures: arm64-v8a, armeabi-v7a
Components: 9 activities, 4 services, 3 receivers, 2 providers

System map

13 boxes



SDKs and libraries

19

Firebase Analytics	Analytics	`com.google.firebase.analytics.FirebaseAnalytics`
Hilt	Architecture	`dagger.hilt.android.HiltAndroidApp`
Jetpack Navigation	Architecture	`androidx.navigation.NavController`

Kotlin Coroutines	Architecture	`kotlinx.coroutines.flow.StateFlow`
WorkManager	Architecture	`androidx.work.PeriodicWorkRequest`
AppsFlyer	Attribution	`com.appsflyer.AppsFlyerLib`
Firebase Crashlytics	Crash and monitoring	`com.google.firebase.crashlytics.FirebaseCrashlytics`
Firebase Performance	Crash and monitoring	`com.google.firebase.perf.FirebasePerformance`
Glide	Images and media	`com.bumptech.glide.Glide`
Google Location	Maps and location	`com.google.android.gms.location.FusedLocationProviderClient`
Google Maps	Maps and location	`com.google.android.gms.maps.GoogleMap`
Moshi	Networking	`com.squareup.moshi.Moshi`
OkHttp	Networking	`okhttp3.OkHttpClient`
Retrofit	Networking	`retrofit2.Retrofit`
Stripe	Payments	`com.stripe.android.paymentsheet.PaymentSheet`
Firebase Cloud Messaging	Push and messaging	`com.google.firebase.messaging.FirebaseMessagingService`
Jetpack Security	Storage	`androidx.security.crypto.EncryptedSharedPreferences`
Room	Storage	`androidx.room.RoomDatabase`
Lottie	UI	`com.airbnb.lottie.LottieAnimationView`

Permissions

8

ACCESS_FINE_LOCATION	**DANGEROUS**	Precise GPS location
ACCESS_COARSE_LOCATION	**DANGEROUS**	Approximate location
CAMERA	**DANGEROUS**	Take photos and video
POST_NOTIFICATIONS	**DANGEROUS**	Show notifications
INTERNET	**normal**	Granted at install
ACCESS_NETWORK_STATE	**normal**	Granted at install
WAKE_LOCK	**normal**	Granted at install
RECEIVE_BOOT_COMPLETED	**normal**	Granted at install

Entry points

5

Every way the app can be started. Each one is the start of at least one flow.

launcher	`com.fooddash.app.ui.SplashActivity`	Opens when the user taps the app icon
deep link	`com.fooddash.app.ui.MainActivity`	fooddash://orders/{id}

deep link	`com.fooddash.app.ui.MainActivity`	https://fooddash.example/r/{restaurant}
push	`com.fooddash.app.push.FoodDashMessagingService`	Handles incoming push messages (FCM)
receiver	`com.fooddash.app.sync.BootReceiver`	BOOT_COMPLETED

Network surface

HOSTS FOUND IN THE CODE

`api.fooddash.example` `cdn.fooddash.example` `track.fooddash.example`

API PATHS

`v1/auth/verify` `v1/auth/refresh` `v1/orders` `v1/orders/{id}` `v1/orders/{id}/confirm` `v1/menus` `v1/restaurants/{id}`

What takes up space

36.6 MB

Code	13.5 MB	37%
Native libraries	9.3 MB	26%
Resources	10.8 MB	29%
Assets	2.5 MB	7%
Other	501 KB	1%

Flows

8 flows · 83 steps

FLOW 1 OF 8 · HAPPY PATH · 11 STEPS

Login with OTP

Phone number and OTP are exchanged for a JWT, which is stored encrypted on the device.

Starts at: LoginActivity Ends with: Home screen with a valid session

01 ● User enters phone number and OTP

User -> Screens · Request · verified

`LoginActivity · activity_login.xml`

02 ● Tapping Continue calls login()

Screens -> ViewModels · Request · inferred

`LoginViewModel.login()`

03 ● ViewModel hands the call to the repository

ViewModels -> Repositories · Request · inferred

`AuthRepository.verifyOtp()`

04 ● POST /v1/auth/verify over HTTPS

Repositories -> API gateway · Request · verified

`AuthApi (Retrofit interface)`

05 ● Gateway routes the call to the auth service

API gateway -> Auth service · Request · inferred

`Inferred from the /auth path prefix`

06 Auth service returns a JWT and a refresh token

Auth service -> API gateway · Response · verified
`TokenResponse`

07 200 OK reaches the app

API gateway -> Repositories · Response · verified
`OkHttpClient`

08 Tokens are saved in encrypted storage

Repositories -> Local storage · Local storage · inferred
`EncryptedSharedPreferences`

09 Repository emits a LoggedIn state

Repositories -> ViewModels · Response · inferred
`StateFlow<AuthState>`

10 App navigates to Home

ViewModels -> Screens · Response · verified
`nav_graph.xml: login -> home`

11 A login_success event is logged

ViewModels -> Analytics · Request · verified
`FirebaseAnalytics.logEvent()`

FLOW 2 OF 8 · HAPPY PATH · 18 STEPS

Place an order

The order is created first, then paid through the payment SDK, then confirmed. The backend sends a push.

Starts at: CartFragment Ends with: Order tracking screen

01 User taps Place order on the cart

User -> Screens · Request · verified
`CartFragment`

02 Cart screen calls placeOrder()

Screens -> ViewModels · Request · inferred
`CartViewModel.placeOrder()`

03 Order request goes to the repository

ViewModels -> Repositories · Request · inferred
`OrderRepository.create()`

04 POST /v1/orders

Repositories -> API gateway · Request · verified
`OrderApi.create()`

05 Gateway routes to the order service

API gateway -> Order service · Request · inferred
`Inferred from the /orders path`

06 Order created with status pending_payment

Order service -> API gateway · Response · verified
`OrderDto.status`

07 201 Created reaches the app

API gateway -> Repositories · Response · verified
`OkHttpClient`

08 ViewModel receives the order id

Repositories -> ViewModels · Response · inferred
`Result<Order>`

09 Payment sheet opens

ViewModels -> Stripe · Request · verified
`PaymentSheet.present()`

10 Payment confirmed by the provider

Stripe -> ViewModels · Response · verified
`PaymentResult.Completed`

11 App confirms the payment

ViewModels -> Repositories · Request · inferred
`OrderRepository.confirm()`

12 POST /v1/orders/{id}/confirm

Repositories -> API gateway · Request · verified
`OrderApi.confirm()`

13 Order service marks the order paid

API gateway -> Order service · Request · inferred
`Inferred`

14 Backend triggers an order-confirmed push

Order service -> Firebase FCM · Request · inferred
`FCM topic: orders`

15 Confirmation returned

Order service -> API gateway · Response · verified
`OrderDto`

16 200 OK reaches the app

API gateway -> Repositories · Response · verified
`OkHttpClient`

17 Order cached for offline viewing

Repositories -> Local storage · Local storage · inferred
`OrderDao.insert()`

18 Tracking screen opens

ViewModels -> Screens · Response · verified
`nav_graph.xml: cart -> tracking`

FLOW 3 OF 8 · HAPPY PATH · 12 STEPS

Push to deep link

A push notification opens a deep link. A cached copy is shown instantly, then refreshed from the server.

Starts at: FoodDashMessagingService Ends with: Order details with live status

01 A push notification arrives

Firebase FCM -> System · Request · verified
`FoodDashMessagingService`

02 User taps the notification

System -> User · Request · verified
`PendingIntent`

03 Deep link fooddash://orders/123 opens order details

System -> Screens · Request · verified
`AndroidManifest intent-filter`

04 Screen requests the order

Screens -> ViewModels · Request · inferred
`OrderDetailViewModel`

05 Repository is asked for order 123

ViewModels -> Repositories · Request · inferred
`OrderRepository.get()`

06 Cached copy shown instantly

Repositories -> Local storage · Local storage · inferred
`OrderDao.getById()`

07 GET /v1/orders/123 for live status

Repositories -> API gateway · Request · verified
`OrderApi.get()`

08 Routed to the order service

API gateway -> Order service · Request · inferred
`Inferred`

09 Latest status returned

Order service -> API gateway · Response · verified
`OrderDto`

10 200 OK reaches the app

API gateway -> Repositories · Response · verified
`OkHttpClient`

11 Fresh data replaces the cache

Repositories -> ViewModels · Response · inferred
`Flow<Order>`

12 Live status is displayed

ViewModels -> Screens · Response · verified
`fragment_order_detail.xml`

FLOW 4 OF 8 · ERROR AND EDGE PATH · 15 STEPS

Session expired

A 401 is caught by an OkHttp authenticator, the token is refreshed and the request retried silently.

Starts at: HomeFragment Ends with: Orders shown without asking the user to log in again

01 Home screen loads past orders

Screens -> ViewModels · Request · inferred
`HomeViewModel.load()`

02 Repository fetches orders

ViewModels -> Repositories · Request · inferred
`OrderRepository.list()`

03 GET /v1/orders with an expired JWT

Repositories -> API gateway · Request · verified
`OrderApi.list()`

04 401 Unauthorized

API gateway -> Repositories · Error · verified
`HTTP 401`

05 Refresh token is read

Repositories -> Local storage · Local storage · inferred
`TokenAuthenticator`

06 POST /v1/auth/refresh

Repositories -> API gateway · Request · verified
`AuthApi.refresh()`

07 Routed to the auth service

API gateway -> Auth service · Request · inferred

`Inferred`

08 New JWT issued

Auth service -> API gateway · Response · verified

`TokenResponse`

09 New token reaches the app

API gateway -> Repositories · Response · verified

`OkHttp Authenticator`

10 Original request retried automatically

Repositories -> API gateway · Request · verified

`OkHttp Authenticator retry`

11 Routed to the order service

API gateway -> Order service · Request · inferred

`Inferred`

12 Orders returned

Order service -> API gateway · Response · verified

`List<OrderDto>`

13 200 OK reaches the app

API gateway -> Repositories · Response · verified

`OkHttpClient`

14 Orders handed to the ViewModel

Repositories -> ViewModels · Response · inferred

`Result<List<Order>>`

15 The user never notices the refresh

ViewModels -> Screens · Response · verified

`fragment_home.xml`

FLOW 5 OF 8 · ERROR AND EDGE PATH · 6 STEPS

Payment declined

The payment SDK reports a decline. The app logs it and offers a retry, and the order stays pending.

Starts at: CheckoutFragment Ends with: Retry dialog

01 User taps Pay

User -> Screens · Request · verified

`CheckoutFragment`

02 Checkout starts the payment

Screens -> ViewModels · Request · inferred

`CheckoutViewModel.pay()`

03 Payment sheet opens

ViewModels -> Stripe · Request · verified

`PaymentSheet.present()`

04 Card declined by the provider

Stripe -> ViewModels · Error · verified

`PaymentResult.Failed`

05 A payment_failed event is logged

ViewModels -> Analytics · Request · verified

`FirebaseAnalytics.logEvent()`

06 Retry dialog is shown

ViewModels -> Screens · Response · verified
`PaymentErrorDialog`

FLOW 6 OF 8 · ERROR AND EDGE PATH · 8 STEPS

Browse while offline

The network call fails, so the repository falls back to the Room cache and marks the data as stale.

Starts at: MenuFragment Ends with: Cached menu with an offline banner

01 User opens the menu with no network

User -> Screens · Request · verified
`MenuFragment`

02 Screen asks for the menu

Screens -> ViewModels · Request · inferred
`MenuViewModel.load()`

03 Repository tries the network first

ViewModels -> Repositories · Request · inferred
`MenuRepository.get()`

04 Request fails: no connection

Repositories -> API gateway · Error · verified
`UnknownHostException`

05 Falls back to the local cache

Repositories -> Local storage · Local storage · inferred
`MenuDao.getAll()`

06 Cached menu returned

Local storage -> Repositories · Response · inferred
`Room database: fooddash.db`

07 Data marked as stale

Repositories -> ViewModels · Response · inferred
`Resource.Cached`

08 Menu shown with an offline banner

ViewModels -> Screens · Response · verified
`view_offline_banner.xml`

FLOW 7 OF 8 · BACKGROUND · 8 STEPS

Nightly menu sync

WorkManager runs a periodic job on unmetered networks to pre-fetch menus into the local database.

Starts at: MenuSyncWorker Ends with: Fresh menu cached before the user opens the app

01 WorkManager starts the periodic job

System -> ViewModels · Request · inferred
`MenuSyncWorker (PeriodicWorkRequest, 12h)`

02 Worker asks the repository to sync

ViewModels -> Repositories · Request · inferred
`MenuRepository.sync()`

03 GET /v1/menus?since={lastSync}

Repositories -> API gateway · Request · verified
`MenuApi.changes()`

04 Routed to the catalogue

API gateway -> Order service · Request · inferred

`Inferred`

05 Changed menus returned

Order service -> API gateway · Response · verified

`List<MenuDto>`

06 200 OK reaches the app

API gateway -> Repositories · Response · verified

`OkHttpClient`

07 Menus upserted into Room

Repositories -> Local storage · Local storage · inferred

`MenuDao.upsertAll()`

08 Sync duration recorded as a trace

Repositories -> Crashlytics · Request · inferred

`FirebasePerformance trace`

FLOW 8 OF 8 · THIRD-PARTY · 5 STEPS

Crash report

An uncaught exception is written to disk and uploaded on the next launch.

Starts at: Any screen Ends with: Crash visible in the crash dashboard

01 An action throws an uncaught exception

Screens -> ViewModels · Request · inferred

`TrackingViewModel.onLocationUpdate()`

02 Crash report written to disk before the app dies

ViewModels -> Local storage · Local storage · inferred

`Crashlytics UncaughtExceptionHandler`

03 User reopens the app

User -> Screens · Request · verified

`SplashActivity`

04 Pending report uploaded

Local storage -> Crashlytics · Request · verified

`Crashlytics upload`

05 Report accepted and deleted locally

Crashlytics -> Local storage · Response · inferred

`Crashlytics`